Programming Learning App (V1)

Authors

Matt ter Steege
m.j.tersteege@students.uu.nl

Max Harmsen
m.harmsen1@uu.nl

October 21, 2026

Contents

1 Class Diagram **2**
2 Design patterns **3**
2.1 Command pattern 3
2.2 Composite pattern 3
2.3 Other patterns and idioms 3
3 Evaluation **4**
3.1 Cohesion and coupling 4
3.2 Is it SOLID? 4
3.3 Requirement changes 4
4 Testing **5**
4.1 Overview and limitations 5
4.2 Interesting foundings 5
5 Work distribution & retrospective **6**
5.1 Work distribution 6
5.2 Retrospective 6
A Mermaid diagrams **7**
A.1 ProgrammingLearningApp UML 7
B Code for Tests **8**

Introduction

During this project, we implemented our design from the first project. The software design has been turned into concrete code implementations. We also made 64 unit tests that test the different functionalities of our program. Our current program accepts one of three hardcoded programs or a custom program that can be inserted through a file input. When a program has been chosen, it shows all moves that the player executed and what the end state of the player is (both the position and the direction the player is facing). Lastly, the program outputs the number of commands the loaded program consists of, the number of repeats the program contains, and the maximum nesting of the program[1].

The link to our GitLab page is: https://git.science.uu.nl/matttersteege/MSO_ProgrammingLearningApp

1 | Class Diagram

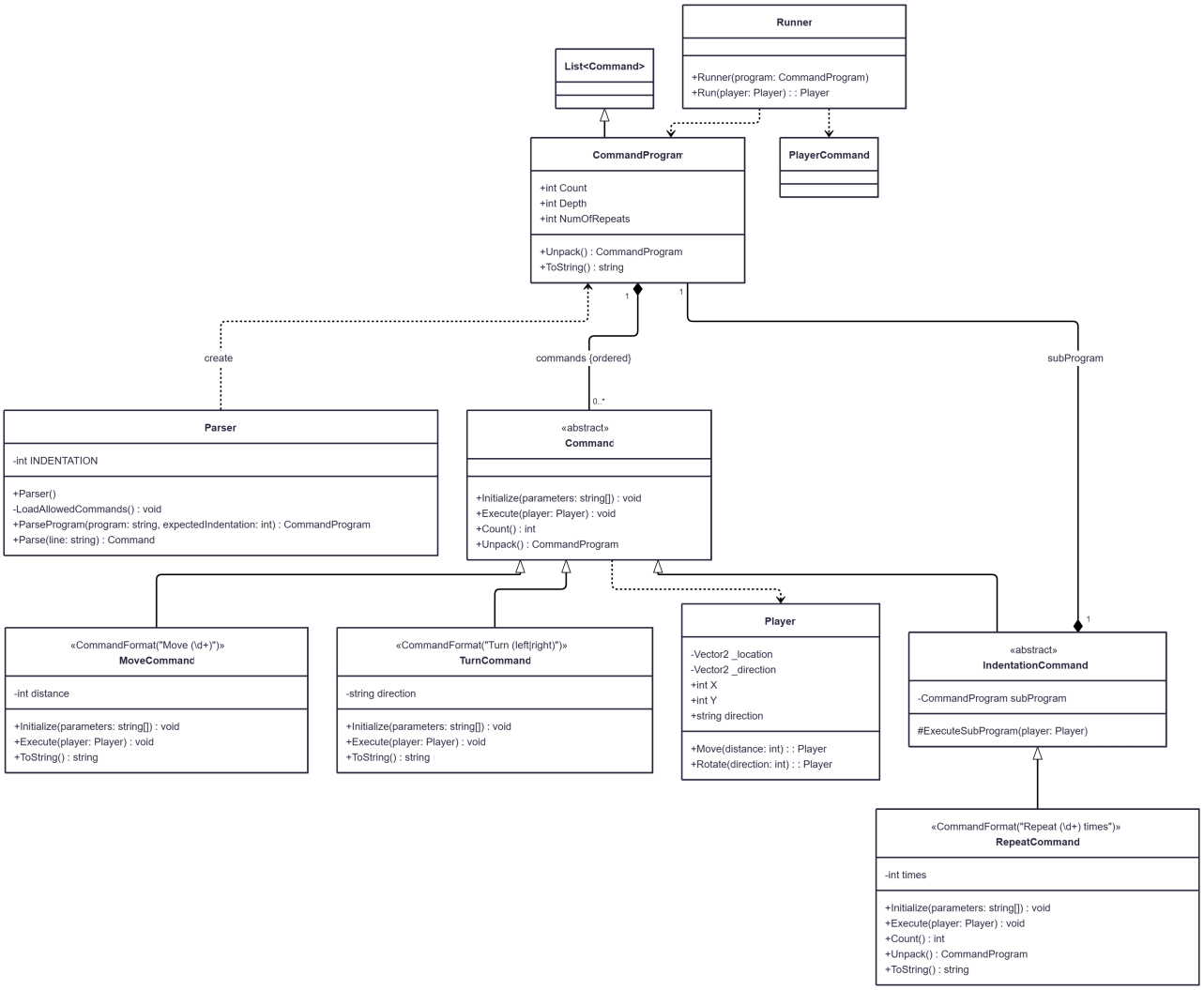


Figure 1: Class diagram for Programming learning app (A.1)

[1]Each repeat will introduce indentation. Repeats in repeats will cause a larger nesting than 1

The diagram can be read as three layers that work together:
- **Representation:** `Command`, `IndentationCommand` and their concrete subclasses (`MoveCommand`, `TurnCommand`, `RepeatCommand`) represent a single instruction. `CommandProgram` is a list of commands and represents a complete (sub)program.
- **Construction:** the `Parser` turns the text a user typed into a `CommandProgram`. It is the only class that knows how text maps to commands.
- **Execution:** the `Runner` executes a `CommandProgram` on a `Player`. The `Player` stores the state of the character (location and direction).

Each of the subclasses of `Command`/`IndentationCommand` (`MoveCommand`, `RepeatCommand`, `TurnCommand`, and its future subclasses) are expected to be decorated with a `CommandFormatAttribute(string format)` attribute. This attribute defines the string format the command should follow, written as a regular expression. For example, the attribute on the `MoveCommand` is `[CommandFormat(@"Move (\d+)")]`. The capture groups of this expression are the arguments that are passed to `Initialize`. For `Move 3` the capture group yields the argument `"3"`, which the `MoveCommand` converts to an integer itself.

2 | Design patterns

Command pattern

`Command` is an abstract base class for all executable instructions. It defines the methods that every command must implement or support. `Initialize(string[] args)` takes a string array that corresponds to the capture groups in the Regex command format defined by the attribute. This way, a variable number of arguments can be passed to a command, making it easy to support commands with different numbers of arguments.

The `Count` method returns the number of commands represented by the instruction, and `Unpack` provides a way to obtain the command program as a flattened list of commands. For example, this allows a `Repeat 4 times` command to be expanded into a list containing the commands that will be executed during those repetitions.

There is also an `IndentationCommand` class which builds on the `Command` class. This class adds a `CommandProgram` member and an `ExecuteSubProgram(Player player)` method, which executes all the commands in the `CommandProgram` member in order. We chose this setup because `RepeatCommand` can delegate the execution of multiple commands to its base class, keeping the repeat part of the command as simple as a `for` loop that calls `ExecuteSubProgram`. Furthermore, this setup allows other types of commands with a body to be created. For example, `if` commands or commands that do not have a predefined endpoint can be implemented, since the command itself decides whether or not the subprogram is executed.

Composite pattern

The composite pattern allows individual objects and groups of object to be handled in a similar way. In our design, this is used by the `CommandProgram` class, which contains a list of `Command` objects. Since each command inherits from the `Command` class, the `CommandProgram` can contain different types of commands without needing to know their specific implementation.

The `RepeatCommand` class builds on this by containing a `CommandProgram` as its subprogram. This allows a repeat command to contain multiple commands, including other repeat commands (thus creating a tree).

One minor note is that this is not a textbook composite pattern, since the CommandProgram is of type `List<Command>` and not of type `Command`.

Other patterns and idioms

Besides the two patterns above, a few smaller ideas are worth mentioning. The `CommandFormat` attribute together with the loading code in the `Parser` acts as a lightweight registry of commands: the set of available commands is discovered through reflection instead of being hard-coded in a `switch` statement. Also, `IndentationCommand` applies the template method idea: the base class provides the shared behaviour for running a body (`ExecuteSubProgram`), while the subclass decides when and how often it is called.

3 | Evaluation

Cohesion and coupling

Cohesion

The classes have a single, clearly defined purpose. The `Player` only stores the character's state (location and direction) and offers the operations to change it (`Move` and `Rotate`). It knows nothing about commands or parsing. `Runner` has exactly one job: running a `CommandProgram` on a `Player`. Each concrete command (`MoveCommand`, `TurnCommand`, `RepeatCommand`) contains only the logic for its own behaviour: how to read its parameters (`Initialize`), how to execute, how to print itself (`ToString`) and, for `RepeatCommand`, how to count and unpack its subprogram. `CommandFormatAttribute` only carries the format of a command, which keeps the syntax of a command next to the class that implements it.

On the other side, `CommandProgram` is not only the container for the commands, but also calculates the metrics, which lowers the cohesion slightly.

Coupling

The coupling between the main parts of the design is low. The `Runner` and `CommandProgram` depend on the abstract `Command`, not on its subclasses. The runner can therefore execute any command, without knowing what kind it is. `Parser` finds commands at runtime through the `CommandFormat` attribute. It does not have to reference `MoveCommand` or `TurnCommand`, so adding a new command does not require changes to the parser. `Command.Initialize(string[])` takes a generic string array, so the parser does not need to know how many or what kind of parameters a command has.

Is it SOLID?

Single responsibility

Most of the classes have only one specific job or task that it executes, we did found some minor issues that we chose to not alter. In the `Parser` class we essentially do 2 different tasks. We (1) load the commands from the assembly using the `CommandFormat` attribute and store them as uninitialized types and (2) Parse the program itself to a `CommandProgram` tree.

Open-Closed

The current command system is setup following the Open-Closed principle. When adding commands, for the 2 styles of commands (indented and singular) you only have to inheret from those classes to obtain all the functionality and the sub-class itself can chose to execute the command or to skip it.

Liskov substitution

As stated before, the `Command` and `IndentedCommand` classes themselves do not contain any logic and would not work in the parser. But during the loading of the command classes, an explicit check is made if the loaded command is an subclass of `Command`, which would fail for the `Command` class itself.

Requirement changes

Conditionals

The current setup of code allows for the easy implementation of conditionals. Since all the command that have a body (I.E. indented commands after said command) and the "parent" has to execute the code to run the code body, so the parent command can choose to not execute because of some state of the program/player.

Player depended

At this moment, the runner is depended on a `Player` object to exist, and this is not good for the coupling. This is why for a good change to the codebase would be removing the Player from the Commands/runner and using the observer pattern to extract the Player from the commands. This would allow for a better system and would also allow for creating variables far easier, while now it would need to be some sort of dictionary on the Player object.

Variables

This brings us to the third requirement change, being support for variables that can store values and be used as arguments in commands. A program could store data using syntax like `set Distance to 3`. This would require an change to the execution of commands.

The current design is not build with this in mind, but could be easily change. This would require a change: adding a `ExecutionContext` class which would like something like this:

```
public class ExecutionContext
{
    public Player Player = new();
    private Dictionary<string, int> variables = new();

    public void SetVariable(string name, int value)
    {
        variables[name] = value;
    }

    public int GetVariable(string name)
    {
        return variables[name];
    }
}
```

This would then be the actual object passed to the Command's execute method and would handle the existence of the Player Object. And instead of executing `player.Move(3)` it would become `context.player.Move(3)`. This way setting and getting variables are done by the command that needs a variable.

4 | Testing

Overview and limitations

For our project, we created a total of 64 unit tests using xUnit.net. The tests are located in `MSOProgrammingLearningApp.Tests` and are divided across 5 files.

We did not write separate tests for `Program.cs`, `ProgrammingException.cs`, and `Runner.cs`.
We left `ProgrammingException.cs` untested because it only contains one straightforward function. The only function within `Runner.cs` is implicitly tested in `RunProgramTests.cs`, so no separate tests for `Runner.cs` are needed.

`Program.cs` is not tested directly because it only contains the static Main function, which initializes the program. Running the program is already covered by `RunProgramTests.cs`, so the only uncovered functions are the translation of the user's input to the file upload to select/get the program. We decided not to rewrite the main function, as this would add unneeded complexity. The functions left untested are straightforward enough to not need tests.

Interesting foundings

Almost all unit tests are quite forward, meaning we will not spend too much attention on those functions. However, during the process of making unit tests, there was a scenario that made us realize that writing tests is quite different from what we are used to, so we wanted to reflect on that.

When we started to write the test functions for `CommandProgramTests`, we noticed that all of the `[Fact]` tests where almost the same, but used different Programs to test on. Coming from "normal" coding, the idea was to combine all different programs in a `[Theory]` block with `[InlineData]` and make 1 test functions. However, this solution doesn't work when one of the arguments requires you to create a new object (in this case `CommandProgram`). To fix this, we came up with the solution shown in Appendix B. The idea stayed the same, creating each test program once, but now save them in `TheoryData<>`. This worked, but now, when the tests ran, we got an fail. And because of using `TheoryData<>`, we didn't know what test program caused that fail, since our IDE only showed it as 1 test. That was when we decided to split everything up again into separate `[Fact]` functions. That way we could derive where the fail came from and use the tests where they were meant to be for.

5 | Work distribution & retrospective

Work distribution

We divided the work for this project equally between the two group members. We both worked on different parts of the assignment, investing approximately the same amount of time and effort. The parts that were developed individually were always reviewed by the other person in order to minimise mistakes. Ultimately, we both contributed equally to the final result.

At the start of the assignment, we established clear agreements regarding deadlines to ensure the collaboration ran as smoothly as possible. The following schedule was used:

Planning

Part	Date finished	Group member
UML diagram	4-10-2026	Matt
Design patterns	5-10-2026	Matt
Assumptions	6-10-2026	Max
Cohesion & coupling	10-10-2026	Matt
Implementation	15-10-2026	Matt + Max
Testing	18-10-2026	Max
Work distribution	18-10-2026	Matt + Max

Retrospective

We had worked together on several assignments before, so we already had a good understanding of each other's expectations. As a result, dividing the tasks went smoothly.

We both put in a lot of effort and had a clear understanding of what to expect from each other in terms of quality, workload, and collaboration. This made the overall process much easier. Any small misunderstanding of last assignment did not happen this project. We are satisfied with how the collaboration went and would not change much about our approach.

A | Mermaid diagrams

ProgrammingLearningApp UML

```
classDiagram
class Command {
    <<abstract>>
    +Initialize(parameters: string[]) void
    +Execute(player: Player) void
    +Count() int
    +Unpack() CommandProgram
}
class IndentationCommand {
    <<abstract>>
    -CommandProgram subProgram
    #ExecuteSubProgram(player: Player)
}
class CommandProgram {
    +int Count
    +Unpack() CommandProgram
    +int Depth
    +int NumOfRepeats
    +ToString() string
}
class MoveCommand {
    <<CommandFormat("Move (\d+)")>>
    -int distance
    +Initialize(parameters: string[]) void
    +Execute(player: Player) void
    +ToString() string
}
class RepeatCommand {
    <<CommandFormat("Repeat (\d+) times")>>
    -int times
    +Initialize(parameters: string[]) void
    +Execute(player: Player) void
    +Count() int
    +Unpack() CommandProgram
    +ToString() string
}
class TurnCommand {
    <<CommandFormat("Turn (left|right)")>>
    -string direction
    +Initialize(parameters: string[]) void
    +Execute(player: Player) void
    +ToString() string
}
class Parser {
    -int INDENTATION
    +Parser()
    -LoadAllowedCommands() void
    +ParseProgram(program: string, expectedIndentation: int) CommandProgram
    +Parse(line: string) Command
}
class Runner {
    +Runner(program: CommandProgram)
    +Run(player: Player) : Player
}
```

```
class Player {
    -Vector2 _location
    -Vector2 _direction
    +int X
    +int Y
    +string direction
    +Move(distance: int) : Player
    +Rotate(direction: int) : Player
}
class List~Command~ {
}
CommandProgram <.. Parser : create
Command <|-- MoveCommand
Command <|-- IndentationCommand
IndentationCommand <|-- RepeatCommand
Command <|-- TurnCommand
List~Command~ <|-- CommandProgram
CommandProgram "1" *-- "0..*" Command : commands {ordered}
IndentationCommand "1" *-- "1" CommandProgram : subProgram
Runner ..> CommandProgram
Runner ..> PlayerCommand
Command ..> Player
```

B | Code for Tests

```
// the program, the expected depth, and the expected number of repeats
public static TheoryData<CommandProgram, int, int> ProgramTestData => new()
 {
     { new CommandProgram(), 0, 0 },
     { new CommandProgram { CreateMove() }, 0, 0 },

     { new CommandProgram{CreateRepeat(CreateMove()),
        CreateRepeat(CreateRepeat(CreateMove())) }, 2, 3 },

     { new CommandProgram { CreateMove(), CreateRepeat(CreateMove()), CreateMove() }, 1, 1 },

     { new CommandProgram { CreateRepeat(CreateMove()), CreateRepeat(CreateMove()) }, 1, 2 }
 };

 [Theory]
 [MemberData(nameof(ProgramTestData))]
 public void TestDepthAndNumOfRepeats(CommandProgram program, int expectedDepth,
    int expectedNumOfRepeats)
 {
     Assert.Equal(expectedDepth, program.Depth);
     Assert.Equal(expectedNumOfRepeats, program.NumOfRepeats);
 }

 public static TheoryData<CommandProgram, int> ProgramUnpackData => new()
 {
     { new CommandProgram(), 0 },
     { new CommandProgram { CreateMove() }, 1 },
     { new CommandProgram { CreateRepeat(CreateRepeat(CreateMove())),
        CreateRepeat(CreateMove()) }, 6 },
     { new CommandProgram { CreateMove(), CreateRepeat(CreateMove()), CreateMove() }, 4 },
     { new CommandProgram { CreateRepeat(CreateMove()), CreateRepeat(CreateMove()) }, 4 }
 };
```

```
[Theory]
[MemberData(nameof(ProgramUnpackData))]
public void TestUnpack(CommandProgram program, int expectedCount)
{
    var unpacked = program.Unpack();
    Assert.Equal(expectedCount, unpacked.Count);

    // All repeat commands are filtered out, so all commands should be MoveCommands
    foreach (var command in unpacked)
        Assert.IsType<MoveCommand>(command);

        Assert.Equal(0, unpacked.Depth);
        Assert.Equal(0, unpacked.NumOfRepeats);
}
```